

LAPORAN TUGAS AKHIR (UAS) PEMROGRAMAN BERORIENTASI OBJEK

*Pengembangan Aplikasi TravelPlanner Berbasis Spring Boot REST API dan React
Ecosystem dengan Penerapan Prinsip SOLID dan Clean OOP*



Disusun Oleh Kelompok 1:

No	Nama	NIM	Peran
1	Yusuf Maulana Arroseyid	25051204426	Lead Developer / Full Stack
2	Reivandani Aji Prakoso	25051204422	Frontend Developer
3	Ayesha Humayra Nadra Rafianti	25051204430	Frontend Developer
4	Daffa Maulana Putra Hanayu	25051204425	Backend Developer
5	Rendy Nur Jamal Prasetyo	25051204435	Backend Developer
6	Celvin Saputra Pratama	25051204439	Frontend Developer
7	Siska Nur Fauziah	25051204440	Frontend Developer

Dosen Pengampu:
Saifudin Yahya, S.Kom., M.T.I.

**PROGRAM STUDI S1 TEKNIK INFORMATIKA
UNIVERSITAS NEGERI SURABAYA
FAKULTAS TEKNIK
TAHUN 2026**

Bab 1 Latar Belakang

1.1 Deskripsi Tema Aplikasi

Aplikasi yang dikembangkan dalam proyek Tugas Besar Ujian Akhir Semester (UAS) ini mengusung tema Manajemen Perjalanan Modern yang diberi nama 'TravelPlanner'. TravelPlanner adalah sebuah aplikasi berbasis web yang dirancang sebagai platform komprehensif untuk memfasilitasi pengguna dalam merencanakan seluruh rangkaian aktivitas perjalanan wisata secara terorganisir dan efisien. Komponen utama dari TravelPlanner berfokus pada tiga pilar utama perjalanan, yaitu: (1) Penjelajahan dan katalogisasi destinasi wisata alam, budaya, dan kuliner, (2) Penyusunan rencana perjalanan harian (Itinerary) yang dinamis, serta (3) Pengelolaan serta kalkulasi anggaran keuangan finansial (Budgeting) yang transparan dan akurat.

Dari segi arsitektural teknik, aplikasi dikembangkan dengan memanfaatkan fleksibilitas arsitektur modern menggunakan kombinasi backend Spring Boot 3.3.4 (Java 21) sebagai core engine REST API, frontend dengan ekosistem React 18 dan Tailwind CSS, serta database PostgreSQL melalui Supabase. Meskipun mengalami penyesuaian media antarmuka dari Java Swing konvensional menjadi Web Application berbasis REST API, struktur kontroler, model data, enkapsulasi, dan pola desain (design patterns) di dalam core engine tetap murni mempertahankan paradigma Pemrograman Berorientasi Objek (PBO) sesuai capaian pembelajaran kurikulum dan mengikuti arsitektur MVC (Model-View-Controller) secara menyeluruh.

1.2 Permasalahan yang Diselesaikan

Melakukan perencanaan perjalanan secara manual sering kali menimbulkan berbagai kendala logistik dan operasional bagi pelancong. Beberapa problem mendasar yang diidentifikasi meliputi:

- a) Fragmentasi Informasi Perjalanan: Pengguna sering kali harus menggunakan platform terpisah untuk mencari referensi tempat wisata, menyusun catatan waktu kunjungan, dan mencatat pengeluaran keuangan.
- b) Ketidakpastian Keuangan (Over-Budgeting): Ketidakmampuan melacak perbandingan antara estimasi alokasi dana awal dengan realisasi pengeluaran aktual di lapangan sering kali mengakibatkan pembengkakan biaya perjalanan tanpa kontrol.
- c) Kompleksitas Manajemen Biaya Bersama: Untuk jenis perjalanan kelompok (Group Trip) atau perjalanan keluarga (Family Trip), perhitungan pembagian porsi biaya per orang atau efisiensi biaya flat secara manual rawan terhadap kesalahan kalkulasi.
- d) Bentrokan Jadwal Destinasi: Penyusunan rencana kunjungan harian tanpa validasi waktu logis sering memicu terjadinya overlap jadwal, di mana dua lokasi wisata berbeda direncanakan pada jam operasi yang saling bertabrakan.

1.3 Tujuan Pembuatan Aplikasi

Berdasarkan rumusan masalah di atas, pembuatan aplikasi TravelPlanner ditujukan untuk:

1. Menyediakan platform satu pintu yang mengintegrasikan pencarian destinasi, pengaturan jadwal harian (itinerary), dan kalkulasi anggaran secara realtime.

2. Mengotomatisasi validasi waktu pengisian jadwal destinasi harian guna mencegah terjadinya tumpang tindih waktu (overlap validation) secara otomatis.
3. Menerapkan pola penghitungan finansial yang cerdas menggunakan pola desain berbasis objek untuk membagi atau menerapkan flat-rate buffer sesuai tipe perjalanan pelancong (Solo, Group, Family).
4. Memenuhi syarat kelulusan mata kuliah PBO melalui implementasi kode nyata berkualitas industri dengan arsitektur MVC, pemanfaatan ORM, dan kepatuhan penuh terhadap prinsip SOLID.

1.4 Penjelasan Singkat Konsep PBO yang Digunakan

Guna menghasilkan kode program yang modular, memiliki reusability tinggi, dan mudah dipelihara (maintainable), aplikasi ini menerapkan empat pilar dasar Pemrograman Berorientasi Objek beserta pola lanjutannya:

- a) Abstraksi (Abstraction): Diterapkan pada deklarasi class abstract seperti Perjalanan, Destinasi, dan AnggaranItem yang bertindak sebagai blueprint dasar yang menetapkan kontrak method abstrak tanpa mengekspos detail internal.
- b) Pewarisan (Inheritance): Mengembangkan spesifikasi model objek dari superclass abstract ke subclass konkret, seperti PerjalananSolo, PerjalananGrup, dan PerjalananKeluarga yang mewarisi seluruh karakteristik dasar class Perjalanan.
- c) Polimorfisme (Polymorphism): Memungkinkan method dengan signature yang sama memberikan hasil kalkulasi dan representasi visual yang berbeda sesuai wujud objek asli di runtime (dynamic binding), seperti implementasi penghitungan biaya per kategori dan pembagian porsi finansial.
- d) Enkapsulasi (Encapsulation): Melindungi integritas data state dengan mendeklarasikan variabel field sebagai private dan menyediakan akses terbatas melalui method getter dan setter bervalidasi ketat (misalnya mencegah nilai biaya negatif atau tanggal selesai mendahului tanggal mulai).
- e) Design Patterns: Menggunakan Factory Pattern untuk standarisasi pembuatan objek kompleks, Strategy Pattern untuk memisahkan algoritma perhitungan budget keuangan yang bervariasi secara dinamis, serta Builder Pattern untuk menyusun konstruksi instansiasi objek.

Bab 2 Implementasi Materi

2.1 Class dan Object

Class merupakan blueprint atau template yang mendefinisikan atribut (state) dan method (behavior) dari suatu entitas objek, sedangkan Object adalah instance konkret dari class tersebut yang mendiami memori runtime. Dalam TravelPlanner, salah satu model dasar fungsional adalah entitas User. Class User mendefinisikan template properti seperti id, nama, email, dan password. Ketika pengguna melakukan proses registrasi atau login, Spring Data JPA secara dinamis menginstansiasi record basis data tersebut menjadi objek Java konkret di dalam memori untuk melayani session.

```
// Contoh Cuplikan Kode Entitas User.java
package com.travelplanner.model;

@Entity
@Table(name = "users")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @NotBlank(message = "Nama tidak boleh kosong")
    private String nama;
    @Column(unique = true)
    private String email;
    private String password;
    // Constructors, Getters, Setters
}
```

2.2 Input dan Output

Proses Input dan Output (I/O) pada TravelPlanner diwujudkan melalui mekanisme pertukaran data berbasis REST API. Input diterima dari sisi client (React) dalam format text JSON (JavaScript Object Notation) melalui Request Body HTTP, yang kemudian dipetakan secara otomatis oleh pustaka Jackson di Spring Boot menjadi objek Java Data Transfer Object (DTO), seperti LoginRequestDTO atau AnggaranRequestDTO. Sebaliknya, proses Output dilakukan dengan mengembalikan ResponseEntity berisi data objek Java yang telah diserialisasi kembali menjadi JSON string menuju HTTP Response client.

```
// Implementasi Input Output Melalui REST Controller
@PostMapping("/login")
public ResponseEntity<AuthResponseDTO> login(@Valid @RequestBody LoginRequestDTO
dto) {
    // Input didapat dari parameter 'dto' yang didekode dari JSON Request
    AuthResponseDTO response = userService.login(dto);
    // Output dikirimkan sebagai JSON dalam bentuk ResponseEntity
    return ResponseEntity.ok(response);
}
```

2.3 Array dan Collection

Aplikasi mengeksplorasi struktur data dinamis Java Collection Framework, khususnya `java.util.List` dan `java.util.ArrayList`, sebagai pengganti array statis konvensional. Penggunaan List memberikan fleksibilitas tinggi karena ukurannya yang elastis (resizable) sesuai penambahan data runtime. Contohnya, objek `HariPerjalanan` memiliki relasi satu-ke-banyak (*One-to-Many*) dengan objek `JadwalDestinasi` yang ditampung secara terurut di dalam variabel `List<JadwalDestinasi> jadwalList`.

```
// Penggunaan List Collection pada Relasi Satu ke Banyak
@Entity
public class HariPerjalanan {
    // ... field lainnya
    @OneToMany(mappedBy = "hariPerjalanan", cascade = CascadeType.ALL, fetch =
FetchType.EAGER)
    @OrderBy("urutan ASC")
    private List<JadwalDestinasi> jadwalList = new ArrayList<>();
}
```

2.4 Searching

Fitur pencarian (*Searching*) diimplementasikan langsung pada layer repositori data menggunakan fungsionalitas Spring Data JPA. Teknik searching memanfaatkan Query Method otomatis dengan kata kunci `ContainingIgnoreCase`, yang di balik layar ditranslasikan menjadi query SQL `LIKE %query%` dengan sifat case-insensitive. Fitur ini sangat vital untuk mencari nama destinasi wisata berdasarkan input teks fleksibel pengguna.

```
// Interface DestinasiRepository.java memanfaatkan JPA Searching
package com.travelplanner.repository;

import com.travelplanner.model.base.Destinasi;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;

public interface DestinasiRepository extends JpaRepository<Destinasi, Long> {
    // Fitur pencarian fleksibel berdasarkan nama tempat
    List<Destinasi> findByNameContainingIgnoreCase(String nama);
}
```

2.5 Sorting

Pengurutan data (*Sorting*) diterapkan untuk menyajikan informasi perjalanan secara kronologis dan logis. Proses sorting diwujudkan melalui dua pendekatan: pengurutan pada level database query menggunakan deklarasi keyword JPA `OrderByUrutanHariAsc` di `HariPerjalananRepository`, serta pengurutan memori runtime memakai Java Stream API dengan `Comparator.comparing` untuk mencari destinasi cover perjalanan paling awal.

```
// Pendekatan Sorting di Level Database dan Runtime Stream
// 1. Database Sorting via Repository
List<HariPerjalanan> findByPerjalananIdOrderByUrutanHariAsc(Long perjalananId);

// 2. Stream Sorting di PerjalananService.java
Optional<JadwalDestinasi> destinasiPertama = jadwalHariHari.stream()
    .min(Comparator.comparing(HariPerjalanan::getTanggal))
    .flatMap(h -> h.getJadwalList().stream().findFirst());
```

2.6 Pewarisan (Inheritance)

Pewarisan diimplementasikan secara ekstensif menggunakan strategi pemetaan ORM `InheritanceType.SINGLE\_TABLE` di mana satu tabel fisik database digunakan bersama oleh seluruh hierarki subclass dengan kolom diskriminator khusus. Superclass abstract seperti `Perjalanan` mewariskan properti inti (`id`, `namaTrip`, `tanggalMulai`) kepada tiga subclass spesifik: `PerjalananSolo` (atribut `moodPerjalanan`), `PerjalananGrup` (`jumlahPeserta`), dan `PerjalananKeluarga` (`jumlahDewasa`, `jumlahAnak`). Konsep serupa diterapkan pada `Destinasi` (turunan `WisataAlam`, `WisataBudaya`, `WisataKuliner`) serta `AnggaranItem`.

```
// Pemetaan Inheritance Pemrograman Berorientasi Objek ke Basis Data
@Entity
@Table(name = "trips")
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name = "tipe_perjalanan", discriminatorType =
DiscriminatorType.STRING)
public abstract class Perjalanan {
    @Id @GeneratedValue private Long id;
    private String namaTrip;
    public abstract String getTipePerjalanan();
}

@Entity @DiscriminatorValue("SOLO")
public class PerjalananSolo extends Perjalanan {
    private String moodPerjalanan;
    @Override public String getTipePerjalanan() { return "Solo Trip"; }
}
```

2.7 Polimorfisme

Polimorfisme dalam TravelPlanner dimanfaatkan untuk mengesekusi logika bisnis kalkulasi keuangan finansial secara dinamis berdasarkan tipe instansiasi objek asli di memori. Method `hitungBiayaPerOrang(double totalBiaya)` pada class `PerjalananSolo` akan mengembalikan 100% nilai biaya, sedangkan pada `PerjalananGrup` membagi rata nilai biaya dengan variabel jumlah peserta, dan pada `PerjalananKeluarga` memberikan diskon flat efisiensi 10% sebelum membagi biaya kepada anggota dewasa. Polimorfisme ini membuat kode program sangat fleksibel tanpa tumpukan kondisional `if-else` yang kaku.

```
// Implementasi Polimorfisme pada Aturan Penghitungan Finansial
// Di dalam PerjalananGrup.java
@Override
public double hitungBiayaPerOrang(double totalBiaya) {
    if (jumlahPeserta == null || jumlahPeserta <= 0) return totalBiaya;
    return totalBiaya / jumlahPeserta;
}

// Di dalam PerjalananKeluarga.java
@Override
public double hitungBiayaPerOrang(double totalBiaya) {
    int totalKeluarga = getMaxPeserta();
    if (totalKeluarga <= 0) return totalBiaya;
    double totalSetelahEfisiensi = totalBiaya * 0.90; // Diskon keluarga 10%
    return totalSetelahEfisiensi / totalKeluarga;
}
```

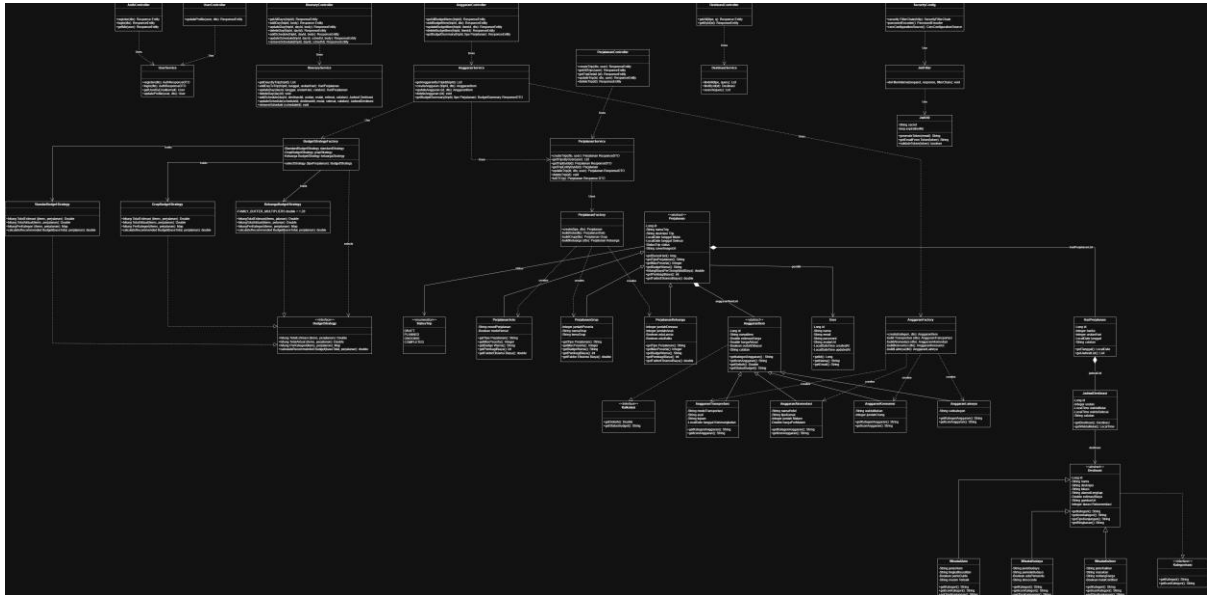
2.8 Enkapsulasi

Prinsip enkapsulasi diterapkan ketat di seluruh model entitas data dengan menetapkan visibilitas seluruh variabel instance member menjadi `private`. Akses modifikasi dan pembacaan data dikontrol penuh oleh public getter dan setter method yang dilengkapi dengan logika validasi internal (data guard) untuk memastikan konsistensi state data objek.

```
// Logika Enkapsulasi Validasi State Data pada Perjalanan.java
public void setTanggalSelesai(LocalDate tanggalSelesai) {
    if (tanggalSelesai != null && this.tanggalMulai != null &&
        tanggalSelesai.isBefore(this.tanggalMulai)) {
        throw new IllegalArgumentException("Tanggal selesai harus setelah tanggal
mulai!");
    }
    this.tanggalSelesai = tanggalSelesai;
}
```

2.9 UML Class Diagram

Perancangan arsitektur kelas aplikasi divisualisasikan menggunakan standar Unified Modeling Language (UML) Class Diagram. Diagram memetakan secara jelas relasi struktural antar objek, batasan pewarisan model abstract, hingga ketergantungan alur data MVC dari Controller menuju Service hingga batas persistensi Repository.



2.10 Design Pattern MVC

Arsitektur source code dipisahkan secara tegas ke dalam tiga lapisan utama guna memenuhi kaidah Separation of Concerns (SoC):

1. Model: Memuat entitas JPA ('User', 'Perjalanan', 'Destinasi', 'AnggaranItem') yang merepresentasikan struktur data bisnis dan tabel database fisik.
2. View: Diwujudkan melalui antarmuka web berbasis komponen React (halaman dashboard, form anggaran, tabel itinerary) yang bertindak sebagai jembatan komunikasi grafis dengan pengguna akhir.
3. Controller: Bertindak sebagai pengatur alur data yang mengekspos endpoint REST API (seperti 'PerjalananController', 'AnggaranController'), menerima request input dari view, mendelegasikan pemrosesan ke layer service bisnis, dan mengembalikan data output.

2.11 Java GUI/Web Improvisation

Sesuai dengan izin dan kebijakan fleksibilitas pengembangan dari dosen pengampu, antarmuka visual aplikasi mengalami transformasi kreatif dari teknologi Java GUI/Swing konvensional menjadi Modern Web Application berbasis arsitektur Single Page Application (SPA). Komponen visual diimplementasikan memakai ekosistem React, Tailwind CSS, dan pustaka ikon Lucide React. Integrasi data visual dihubungkan secara dinamis ke REST Controller Java Spring Boot melalui media library Axios dengan menyertakan token otentikasi JWT (JSON Web Token) pada header HTTP Authorization di setiap transaksi data.

2.12 Organisasi Source Code Project

Struktur direktori pengorganisasian berkas source code diatur rapi berbasis package Java standard:

```
TravelPlanner/
├── backend/
│   ├── src/main/java/com/travelplanner/
│   │   ├── config/          # SecurityConfig, JwtFilter, CorsConfig
│   │   ├── controller/      # REST Controllers (Auth, Trip, Budget, dll)
│   │   ├── dto/
│   │   │   ├── request/     # DTOs untuk input (LoginRequestDTO, dll)
│   │   │   └── response/     # DTOs untuk output (PerjalananResponseDTO, dll)
│   │   ├── exception/       # Custom Exceptions & GlobalExceptionHandler
│   │   ├── factory/         # PerjalananFactory, AnggaranFactory
│   │   ├── model/
│   │   │   ├── base/        # Abstract classes (Perjalanan, Destinasi, AnggaranItem)
│   │   │   ├── perjalanan/  # Subclasses (Solo, Grup, Keluarga)
│   │   │   └── destinasi/    # Subclasses (WisataAlam, WisataBudaya, WisataKuliner)
│   │   │
│   │   └── anggaran/        # Subclasses (Transportasi, Akomodasi, Konsumsi, Lainnya)
│   │
│   │   ├── enums/           # StatusTrip enum
│   │   ├── repository/      # JPA Repositories
│   │   ├── service/         # Business Logic Services
│   │   ├── strategy/        # Budget Strategy Pattern
│   │   └── util/            # JwtUtil, DataSeeder
│   └── src/main/resources/
│       └── application.properties
├── frontend/
│   └── src/
│       ├── components/
│       │   ├── budget/      # BudgetTable, BudgetForm, BudgetSummaryChart
│       │   ├── common/      # Navbar, Footer, LoadingSpinner, ProtectedRoute
│       │   ├── destination/  # DestinationCard, CategoryFilter
│       │   ├── itinerary/    # DayCard, ScheduleItem, DestinationScheduleModal
│       │   └── trip/         # TripCard, TripTypeSelector, StatusBadge, EditTripModal
│       │
│       ├── context/         # AuthContext (global state management)
│       ├── pages/           # Halaman-halaman utama aplikasi
│       ├── services/        # API service functions (tripService, budgetService, dll)
│       │
│       └── utils/           # formatDate, formatCurrency
└── .gitignore
```

Bab 3 Hasil Implementasi

3.1 Hasil Running Aplikasi & Dokumentasi GUI

Aplikasi telah diuji dan berjalan stabil di lingkungan development lokal dengan port backend 8080 dan port frontend Vite 5173. Pengguna dapat mendaftarkan akun baru, melakukan otentikasi login, menjelajahi database destinasi wisata, menyusun agenda hari per hari, serta menambahkan rincian item alokasi anggaran dana.

3.1.1 Landing Page

Halaman utama menampilkan video background perjalanan, tagline 'Satu Rencana. Seribu Kenangan.' dengan efek scroll parallax, dan navigasi ke halaman register/login.



ITINERARY HARIAN

Susun jadwal perjalanan Anda, hari demi hari.

Tambahkan destinasi ke setiap hari perjalanan, atur waktu kunjungan, dan lihat seluruh jadwal Anda tersaji dengan rapi. Dari wisata alam, wisata budaya, hingga wisata kuliner. Semua bisa dipadukan dalam satu itinerary.



MANAJEMEN ANGGARAN

Kelola budget secara transparan, tanpa khawatir.

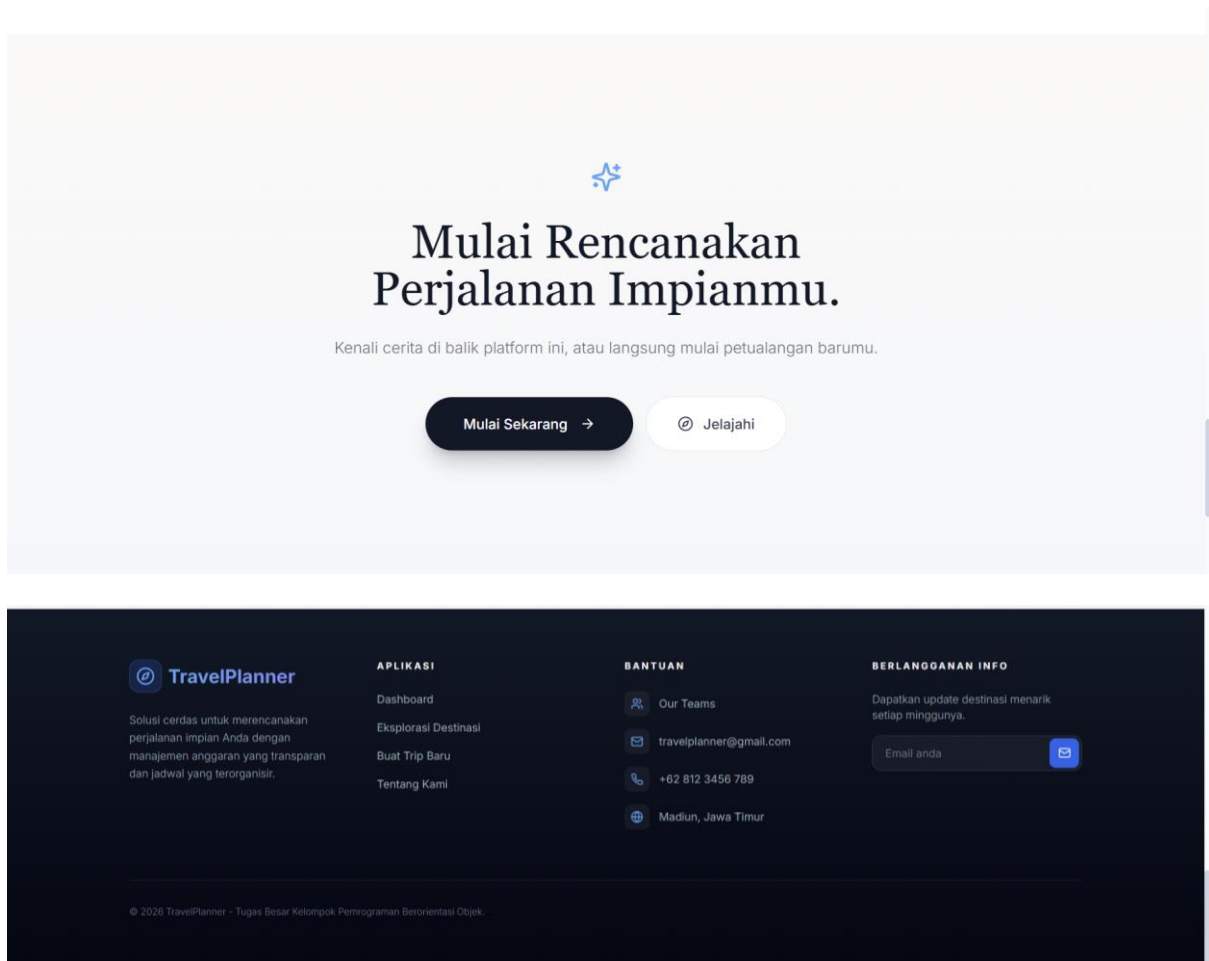
Catat estimasi dan pengeluaran aktual untuk transportasi, akomodasi, konsumsi, dan lainnya. Trip solo, grup, atau keluarga. Anggaran dihitung sesuai kebutuhan masing-masing jenis perjalanan.

BERBAGAI JENIS PERJALANAN

Solo, grup, atau keluarga. Kami mengerti kebutuhan Anda.

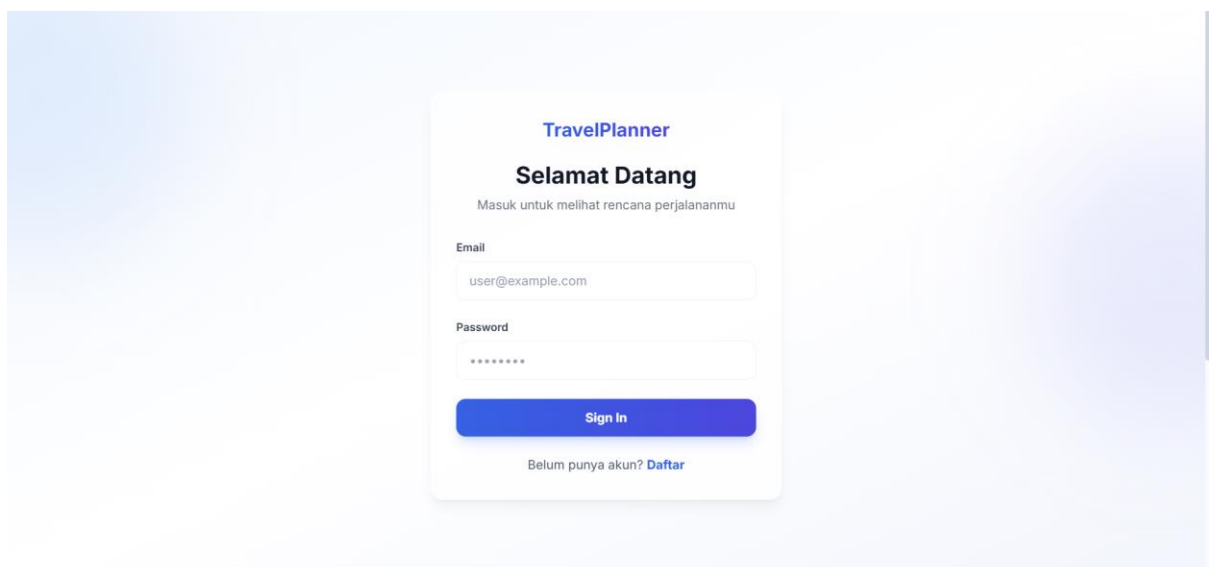
Setiap tipe perjalanan memiliki keunikannya masing-masing. Trip solo untuk momen healing pribadi, trip grup untuk petualangan bersama sahabat, atau trip keluarga dengan perhitungan khusus.





3.1.2 Login Page

Form registrasi dan login dengan glass morphism design. Validasi input dilakukan di frontend (HTML5) dan backend (Bean Validation). JWT token disimpan di localStorage setelah login berhasil.



3.1.3 Register Page

TravelPlanner

Buat Akun

Gabung sekarang dan mulai petualanganmu

Nama Lengkap

siska f

Email

siskaa@gmail.com

Password

.....

Daftar Sekarang

Sudah punya akun? [Masuk](#)

3.1.4 Dashboard

Menampilkan semua trip milik user dalam bentuk grid TripCard. Setiap card menampilkan tipe perjalanan, status badge, destinasi utama, tanggal, dan total estimasi anggaran.

TravelPlanner

Dashboard

Destinations

Halo, Wong kok unik!

Dashboard Saya

Halo Wong kok unik, Anda memiliki 12 rencana perjalanan.

+ Rencana Baru

Solo Trip

Sedang Liburan

Libur Semester

Hari ini: Gudeg Yu Djum

WAKTU

26 Mei 2026

ANGGARAN

Rp 21.500

Lihat Detail Rencana →

Group Trip

Selesai

Cinta Tulus Suci

Gunung Bromo

WAKTU

18 Mei 2026

ANGGARAN

Rp 700

Lihat Detail Rencana →

Solo Trip

Selesai

Trip ke Gunung Bromo

Gunung Bromo & 2 lainnya

WAKTU

14 Mei 2026

ANGGARAN

Rp 600

Lihat Detail Rencana →

Solo Trip

Selesai

LIBUR asek

Kawah Ijen & 3 lainnya

WAKTU

6 Mei 2026

ANGGARAN

Rp 550.000

Family Trip

Direncanakan

tahu

Belum ada destinasi

WAKTU

25 Juli 2028

ANGGARAN

Rp 0

Solo Trip

Sedang Liburan

Holiday

Air Terjun Tumpak Sewu & 3 lainnya

WAKTU

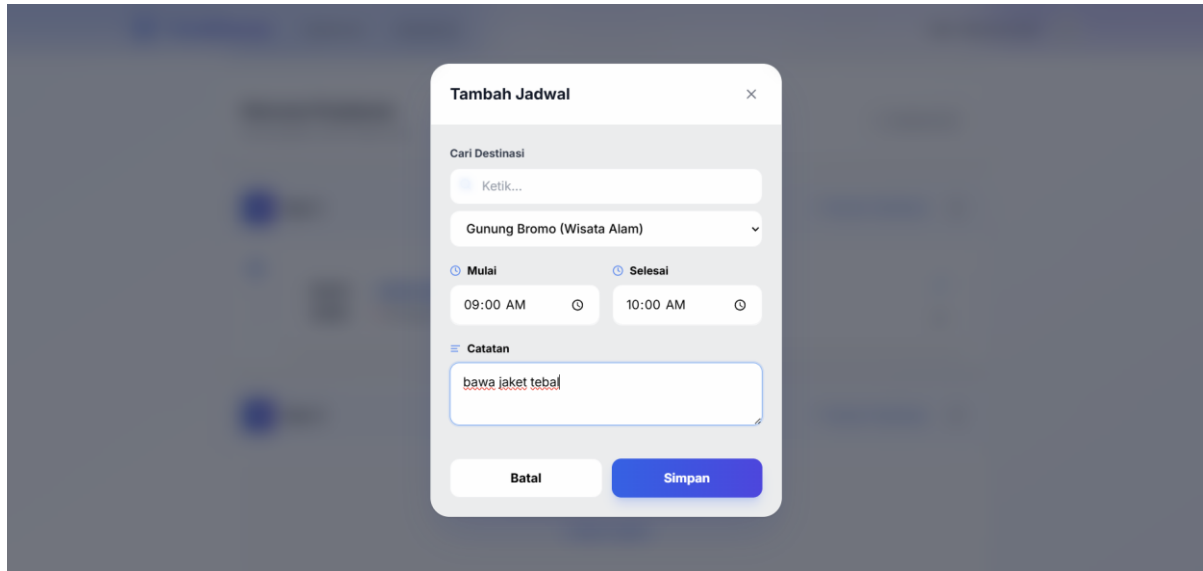
26 April 2026

ANGGARAN

Rp 370.000

3.1.5 Buat Trip Baru

Form pembuatan trip dengan selector tipe perjalanan (Solo/Grup/Keluarga). Field tambahan muncul secara kondisional berdasarkan tipe yang dipilih (contoh: jumlah peserta untuk Grup).

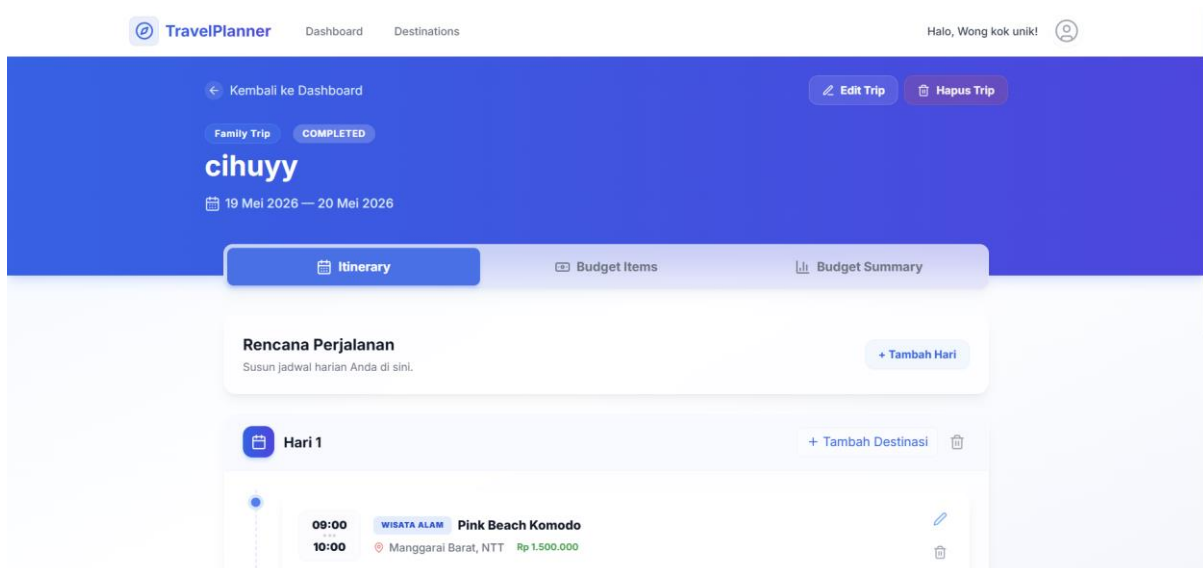


The screenshot shows a modal titled "Tambah Jadwal" (Add Schedule) with a close button (X) in the top right corner. The form contains the following fields and controls:

- Cari Destinasi** (Search Destination): A text input field with a placeholder "Ketik..." (Type...).
- Gunung Bromo (Wisata Alam)**: A dropdown menu showing the selected destination.
- Mulai** (Start): A time selector showing "09:00 AM".
- Selesai** (End): A time selector showing "10:00 AM".
- Catatan** (Notes): A text area with the note "bawa jaket tebal" (bring a thick jacket).
- Batal** (Cancel) and **Simpan** (Save) buttons at the bottom.

3.1.6 Trip Detail – Tab Itinerary

Menampilkan daftar hari perjalanan dalam bentuk DayCard dengan timeline jadwal kunjungan destinasi. Pengguna dapat menambah hari, menambah jadwal destinasi, serta mengedit dan menghapus jadwal.



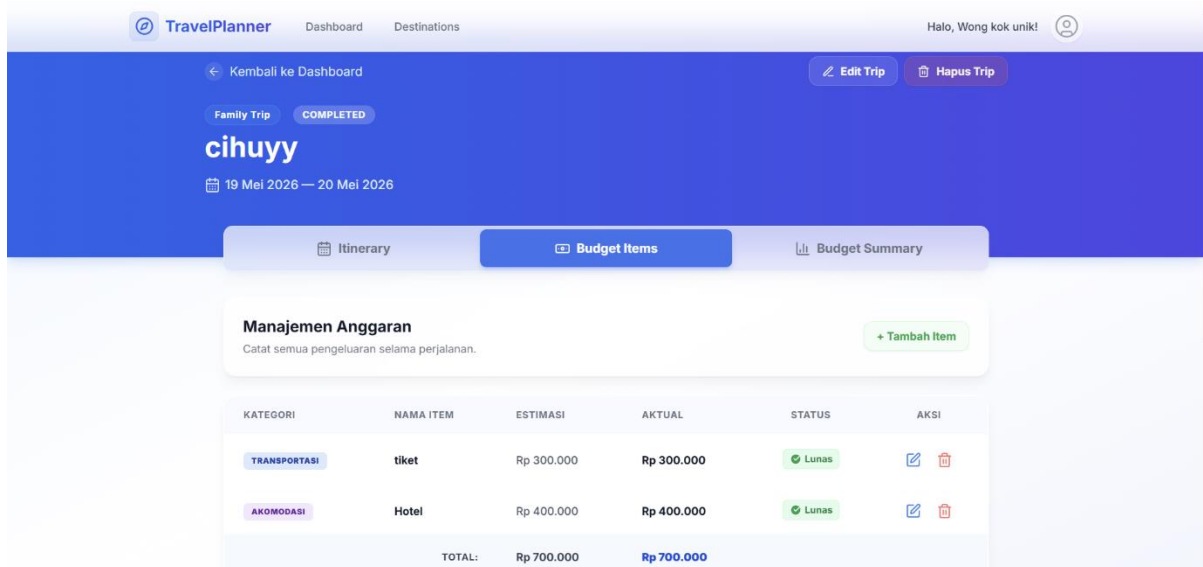
The screenshot shows the "TravelPlanner" app interface. The top navigation bar includes "Dashboard" and "Destinations". The user is logged in as "Halo, Wong kok unik!". The main content area shows the "Itinerary" tab for a trip named "cihuyy" (Family Trip, COMPLETED) scheduled from "19 Mei 2026" to "20 Mei 2026". The "Itinerary" tab is active, showing a "Rencana Perjalanan" (Travel Plan) section with a "+ Tambah Hari" (Add Day) button. Below this, the "Hari 1" (Day 1) section is visible, showing a timeline of activities:

- 09:00**: WISATA ALAM Pink Beach Komodo
- 10:00**: Manggarai Barat, NTT (Rp 1.500.000)

There are buttons for "+ Tambah Destinasi" (Add Destination) and a trash icon for deleting the day.

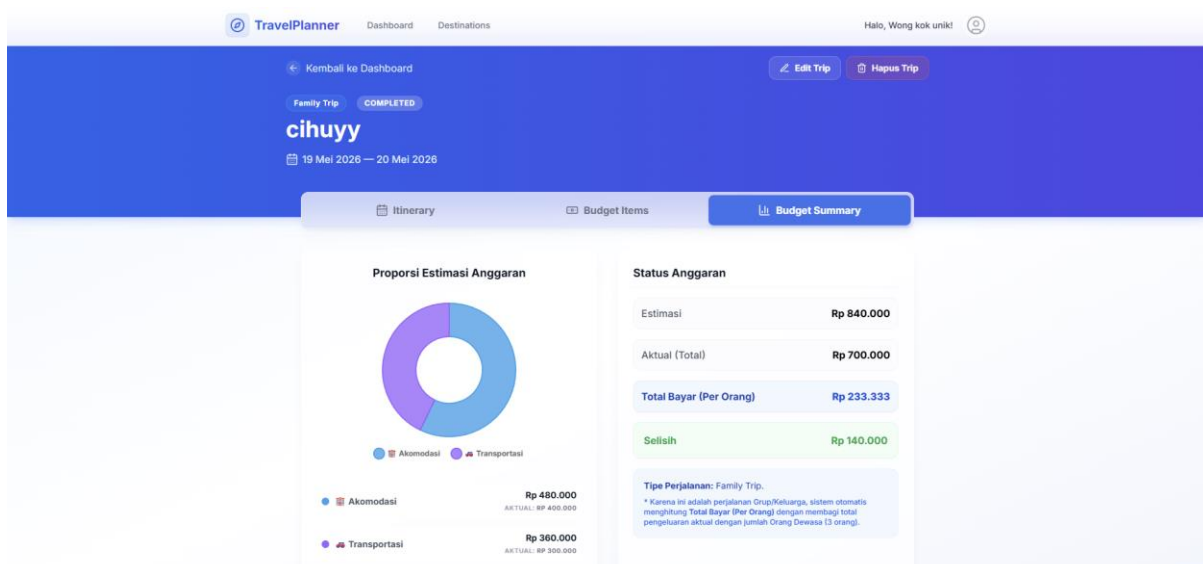
3.1.7 Trip Detail – Tab Budget

Menampilkan tabel anggaran dengan kategori (Transportasi, Akomodasi, Konsumsi, Lainnya), estimasi, aktual, dan status pembayaran. Pengguna dapat menambah, mengedit, dan menghapus item anggaran.



3.1.8 Trip Budget Summary

Menampilkan pie chart (Doughnut Chart) proporsi estimasi anggaran per kategori, total estimasi vs aktual, selisih, dan informasi biaya per orang (untuk trip Grup/Keluarga).



3.2 Penjelasan Fitur Utama

Aplikasi TravelPlanner menyediakan serangkaian fitur fungsional mutakhir sebagai berikut:

- Otentikasi Stateless JWT: Sistem keamanan berlapis yang mendaftarkan dan memvalidasi session pengguna menggunakan enkripsi token JWT yang aman.
- Katalog Eksplorasi Destinasi: Membaca database seeder destinasi yang dikelompokkan ke dalam kategori Alam, Budaya, dan Kuliner lengkap dengan rincian biaya tiket serta tips kunjungan.
- Validasi Batas Durasi Rencana: Itinerary Service secara otomatis menolak penambahan objek hari perjalanan baru jika kuantitas hari telah melewati durasi rentang tanggal yang didefinisikan pada informasi dasar trip.
- Validasi Waktu Jadwal Bertabrakan (Overlap Validation): Ketika pengguna menambahkan agenda destinasi baru pada hari yang sama, sistem secara cerdas melakukan komparasi interval waktu guna mendeteksi dan menggagalkan input jika waktu mulai dan selesai bertabrakan dengan jadwal eksis.
- Polimorfisme Finansial & Chart Donat Anggaran: Menyajikan visualisasi proporsi pemakaian dana per kategori, ringkasan selisih budget (Over/Under Budget Status), serta pembagian porsi tagihan pembayaran per orang otomatis bagi perjalanan tipe Grup atau Keluarga.

3.3 Kendala dan Solusi Implementasi

Kendala / Hambatan Teknis	Solusi / Langkah Penyelesaian
Terjadi galat (crash error) Jackson Serialization sewaktu melakukan parsing objek JSON untuk relasi lazy fetching Hibernate.	Mengubah konfigurasi tipe fetch data pada anotasi JPA ManyToOne dari FetchType.LAZY menjadi FetchType.EAGER pada berkas JadwalDestinasi.java dan HariPerjalanan.java agar relasi data dimuat instan sewaktu serialisasi.
Terjadi error Cross-Origin Resource Sharing (CORS) ketika antarmuka React mencoba menembak data menuju REST API Spring Boot.	Membuat berkas konfigurasi CorsConfig.java dan menyematkan anotasi @CrossOrigin("*") pada seluruh layer REST Controller guna mengizinkan origin lokal frontend (localhost:5173).
Pengguna dapat memasukkan jadwal kunjungan destinasi wisata yang saling bertabrakan atau tumpang tindih waktu (<i>overlap schedule</i>) pada hari yang sama.	Menambahkan logika <i>Overlap Validation</i> pada berkas ItineraryService.java di dalam method addSchedule(). Sistem melakukan iterasi terhadap jadwalList eksis pada hari tersebut, lalu membandingkan rentang waktu menggunakan kondisi mulai.isBefore(adaSelesai) && selesai.isAfter(adaMulai). Jika bertabrakan,

	sistem membatalkan penyimpanan dan melempar <code>InvalidInputException</code> .
--	--

3.4 Tautan Manajemen Proyek & Dokumentasi Video

Seluruh perkembangan manajemen berkas proyek dan pemaparan sistem dapat diakses secara terbuka melalui tautan berikut:

- a) Link Repository GitHub Proyek:
<https://github.com/yzffm/Project-UAS-PBO>
- b) Link Dokumentasi Video Presentasi YouTube:
<https://youtu.be/w8mLRzxtmsQ?si=mep9XJSPPwciEzXH>